Tomathics — Documento de Concepção

Disciplina Mobile 2 · Engenharia de Software · Versão 0.1 (rascunho para alinhamento) · 30 de outubro de 2026

Como ler este documento

Este documento reúne tudo o que foi definido e conversado até aqui sobre o Tomathics. Como o projeto está em fase de ideação, cada ponto relevante traz uma etiqueta de status:

- **[Definido]**: decisão ou descrição vinda diretamente do autor da ideia.
- **[Proposto]**: sugestão feita durante a conversa e adotada como hipótese de trabalho, a confirmar.
- **[Em aberto]**: decisão pendente, reunida na seção 13.

Por decisão do autor, a ordem do trabalho é concepção e documentação primeiro; tecnologia e desenvolvimento ficam por último. Por isso a seção de arquitetura registra apenas restrições e perguntas, sem escolher ferramentas.

1. Visão geral

1.1 O que é o Tomathics [Definido]

O Tomathics é uma ferramenta de automação de tela com editor visual baseado em nós, no estilo dos Blueprints da Unreal Engine. Nele, o usuário descreve cenas, condições e ações de qualquer aplicação (um programa do Windows, um jogo em um emulador Android ou um site no navegador), e um motor executa essas automações no PC.

O que o separa de um gravador de macros com nós é uma segunda camada, inspirada no Applied Energistics 2 (AE2), mod do Minecraft. No AE2, depois de ensinar ao sistema que tronco vira madeira e madeira vira graveto, basta pedir um graveto: se faltar madeira e houver tronco, ele inclui a etapa intermediária sozinho. No Tomathics, cada automação declara o que precisa para rodar e o que entrega, e o motor descobre quais automações precisam rodar antes para destravar o que foi pedido.

1.2 Problema [Proposto]

Automações de tela tradicionais (macros, gravadores de cliques) são sequências lineares: cada tarefa nova exige religar à mão tudo o que vem antes. Quando o cenário cresce, com muitos itens, várias máquinas, tempos de espera e recursos limitados, esse trabalho

manual não escala. Além disso, a tela é uma fonte de informação incerta (pode atrasar, mudar de cena sozinha ou exibir pop-ups), o que deixa frágil qualquer automação que apenas repete passos.

1.3 Objetivos

Objetivo geral [Definido]. Permitir que o usuário ensine receitas de automação uma única vez e depois peça apenas o resultado final, deixando o sistema planejar, executar e monitorar o caminho.

Objetivos específicos:

- Oferecer um editor visual de nós para descrever cenas, condições e ações. [Definido]
- Permitir que as automações se encadeiem sozinhas a partir de contratos (o que precisam e o que entregam), como no AE2. [Definido]
- Oferecer um monitor para acompanhar o que está em execução, o estado de cada etapa e quanto falta. [Definido]
- Manter rotinas contínuas para que nada fique parado. [Definido]
- Funcionar de forma independente do aplicativo automatizado. [Definido]
- Disponibilizar o editor em Windows, navegador e Android, com execução somente no PC. [Definido]

1.4 Contexto acadêmico [Definido]

O Tomathics será desenvolvido na disciplina Mobile 2 do curso de Engenharia de Software. O aplicativo Android é uma das plataformas do editor. Seu papel adicional como monitor e controle remoto do motor é uma proposta (seção 4.6).

1.5 Escopo

Dentro do escopo [Definido]: editor visual de nós; receitas com contrato; resolução automática de dependências; execução no PC; monitor de objetivos; rotinas contínuas; uso em vários alvos.

Fora do escopo:

- Execução das automações no celular: o Android edita (e, possivelmente, monitora), mas não executa. [Definido]
- Gravador de macros linear, sem planejamento. [Proposto]
- Produto dedicado a um único jogo: o Hay Day é apenas estudo de caso. [Proposto]
- Ferramenta para contornar termos de uso ou sistemas anti-trapaça (ver seção 9). [Proposto]

2. Plataformas e alvos

Componente	Onde roda	Status
Editor visual	Windows, navegador e Android	Definido
Motor de execução (planejamento, execução e leitura da tela)	Somente em PC	Definido
Monitor e controle remoto	Navegador e Android, falando com o motor	Proposto
Alvos automatizados	Programas do Windows, emulador Android no PC e abas de navegador	Definido

Alvo [Proposto]. É a peça que diz onde o Tomathics enxerga e age: uma janela de programa, um emulador ou uma aba de navegador. Cenas e entidades pertencem a um alvo, e planejador, monitor e rotinas funcionam igual para qualquer um. Hay Day no emulador, um programa do Windows e um site são três alvos do mesmo sistema. Como cada alvo é lido e controlado por baixo fica para a fase de tecnologia.

O sistema operacional do motor é presumido como Windows; outros sistemas ficam em aberto (seção 13).

3. Modelo conceitual

3.1 Duas camadas [Proposto]

Camada	Pergunta que responde	O que contém
Fluxo	Como fazer?	Grafo de nós ligado à mão: verificar cena, agir conforme o estado, esperar até um estado, supervisionar outras automações
Planejamento	O que é preciso antes?	Contratos das receitas (o que precisa e o que entrega); o motor encadeia sozinho

Cada automação é uma **receita**: um mini-programa visual por dentro (o fluxo) e um contrato por fora. O planejador enxerga apenas o contrato. É isso que faz uma receita nova virar peça disponível para todas as outras, sem religar nada.

3.2 Analogia com o AE2 [Definido; nomes propostos]

AE2	Tomathics	Significado
Item e quantidade	Estado	Cena atual, quantidades (300 de ouro), vagas livres, tempos
Pattern	Receita	Automação com contrato
Pedido de craft	Objetivo	Resultado pedido, com fim (ex.: 10 sanduíches)
Plano de craft	Plano	Receitas na ordem certa, com o que gasta e o que falta
Crafting CPU	Executor	Quem executa o plano
Crafting Status	Monitor de objetivos	Progresso do que está em execução
“Faltam itens”	Dependência não resolvida	Falta um recurso ou uma receita que o produza

3.3 Glossário [Proposto]

Termo	Definição	Exemplo
Alvo	Janela, emulador ou aba de navegador onde o sistema observa e age	Emulador com Hay Day
Cena	Situação reconhecível da tela, representada por um bloco no editor	Fazenda, Menu, Batalha
Entidade	Elemento reconhecível numa cena, sobre o qual se pode agir	Padaria, banca
Localizador	Descrição de como achar uma entidade pela aparência, e não por coordenada fixa	Recorte da Padaria + cena em que aparece
Estado	Tudo o que o sistema observa: cena atual, quantidades, vagas e tempos. É lido da tela, não “sabido”	3 trigos; Padaria ocupada, pronta em 3:20
Receita	Automação com fluxo (nós) por dentro e contrato por fora	Fazer pão

Termo	Definição	Exemplo
Contrato	Parte externa da receita: o que consome, o que ocupa, o que entrega e onde acontece	Consome 3 trigos; entrega 1 pão
Objetivo	Pedido com início e fim	Fazer 10 sanduíches
Rotina	Automação contínua, com gatilho e ação, que não tem fim e fica ligada até ser pausada	Colher e vender trigo; coletar moedas
Plano	Receitas na ordem certa, montadas pelo planejador para cumprir um objetivo	Cultivar trigo → Fazer pão → Montar sanduíche
Executor	Componente que executa o plano e age no alvo	—
Vigia	Observador contínuo de uma cena ou valor	Avisar quando o trigo amadurecer
Reserva	Separação de recursos para um plano, invisível às rotinas	30 trigos reservados para o Objetivo #12
Dependência não resolvida	Falta de um recurso ou de uma receita que o produza	Preciso de Queijo, mas nenhuma receita entrega Queijo
Ociosidade	Tempo em que uma entidade está parada, sem trabalho	Padaria parada há 4 minutos

3.4 Efeitos de uma receita sobre os recursos [Proposto]

- **Consome:** o item some (os 3 trigos do pão).
- **Ocupa:** prende um recurso e o devolve ao fim (a vaga da Padaria, até o pão ficar pronto).
- **Entrega:** produz algo novo (1 pão).

Receitas também têm duração: fazer pão não termina no toque, e sim quando o pão fica pronto. O motor inicia a receita, anota quando ela termina e aproveita o intervalo para executar outras.

Exemplo de receita com contrato:

```
Receita: Fazer pão
  Consome:  3 Trigo
  Ocupa:    1 vaga da Padaria, até ficar pronto
  Entrega:  1 Pão
  Onde:     cena “Fazenda” · entidade “Padaria”
  Fluxo:    grafo de nós (abrir → escolher → conferir)
```

3.5 Como o planejador raciocina [Proposto]

O planejador usa **planejamento regressivo**: parte do objetivo e vai para trás. Para cada necessidade, confere o estado atual; se faltar algo, procura uma receita que entregue aquilo e repete o processo. É o mesmo tipo de problema da literatura de planejamento por pré-condições e efeitos (STRIPS) e de sua variante GOAP, usada em IA de jogos. Ambos servem de base teórica.

```
Objetivo: 1 Sanduíche
├─ precisa de Pão → receita “Fazer pão”
|  ├─ 3 Trigo → tenho? senão, “Cultivar trigo”
|  └─ 1 vaga na Padaria → livre? senão, esperar
└─ (outros ingredientes, cada um com seu ramo)
```

Regras do raciocínio:

- A receita do sanduíche só diz “preciso de Pão” e nem sabe que “Fazer pão” existe. Quem liga uma na outra é o planejador. Por isso o sistema cresce sem religação: cada receita nova vira uma peça disponível para todas as outras.
- As quantidades são somadas: se o sanduíche pedisse 2 pães, o motor já somaria 6 trigos.
- Antes de executar, o motor mostra o plano: o que será feito, o que será gasto e o que está faltando (como a tela de confirmação do AE2).
- Receitas inversas (tronco ↔ madeira) formam ciclo: o planejador precisa detectar e evitar loops.
- Com poucas vagas, um pedido grande vira várias rodadas, e o plano estima isso.
- Se o estado observado divergir do esperado, o motor replaneja a partir do que realmente vê.

4. Telas e experiência de uso

As três telas abaixo foram definidas pelo autor; os detalhes internos de cada uma são propostas a validar.

4.1 Visão geral

Tela	Função	Referência	Status
Biblioteca e editor de nós	Ver as receitas já ensinadas e os processos possíveis; criar e editar o fluxo	Terminal do AE2 e Blueprints	Definido
Monitor de objetivos	Acompanhar as tarefas em execução, o estado em que estão e quanto falta	Crafting Status do AE2	Definido
Rotinas	Ver o que está sendo feito para que nada fique parado	—	Definido

4.2 Biblioteca e editor de nós

Dois níveis de zoom [Proposto]:

1. **Mapa de receitas.** Desenhado pelo próprio motor, ligando o que cada receita entrega ao que outras consomem (Trigo → Pão → Sanduíche). O que não tem produtor aparece em vermelho, com ícone de alerta (“preciso de Queijo, mas nenhuma receita entrega Queijo”). Isso responde ao caso de uma ação que ainda não pode ser feita porque falta algo e exige uma automação anterior. [Definido]
2. **Editor de nós.** Ao abrir uma receita, aparece o fluxo dela, no estilo dos Blueprints, com blocos que representam cenas e cenários. [Definido]

4.3 Monitor de objetivos

Responde à pergunta “por que isso ainda não terminou?”. Exemplo com números ilustrativos:

```
Objetivo #12 · 10 Sanduíches          0/10 · ~25 min
├─ Pão ×10                            produzindo · 0/10
│  ├─ Trigo ×30                       reservado
│  └─ Padaria                         ocupada, pronto em 3:20
├─ Outros ingredientes                em estoque
└─ Montar sanduíches                  esperando o Pão
Última leitura da tela: há 40 s
```

O monitor mostra o progresso e a estimativa, a árvore de dependências com o estado de cada etapa e o que cada uma aguarda. A última linha é proposital: como o estado é lido da tela, o monitor deve admitir a idade da informação. Ações previstas: pausar, cancelar e ver o plano. [Proposto]

4.4 Rotinas

Uma **rotina** não tem fim e fica ligada até ser pausada, ao contrário de um objetivo, que sai da lista quando termina. Cada rotina é um gatilho mais uma ação. Exemplos:

- Quando o trigo amadurecer, colher.
- Quando a banca vender, coletar as moedas.
- Manter o trigo entre 30 e 100 (parecido com a manutenção de estoque no AE2).

O caso original do autor: deixar uma automação colhendo e vendendo trigo na banca e depois pegando as moedas. [Definido] A tela mostra o estado de cada rotina (ativa, pausada, esperando), a última execução e a **ociosidade**, ou seja, há quanto tempo cada entidade está parada. É a forma concreta de medir o "não deixar parado". [Proposto]

4.5 Telas de apoio [Proposto, a detalhar]

- **Alvos, cenas e entidades:** cadastrar alvos, capturar a aparência das entidades e organizar cenas.
- **Pré-visualização do plano:** confirmar o que será feito, o que será gasto e o que falta antes de executar.
- **Conexão com o PC:** parear o app com o motor.
- **Notificações:** avisos de conclusão e de problemas.

4.6 Distribuição por plataforma [Proposto]

O editor de nós é mais confortável no PC. Monitor e Rotinas fazem mais sentido no celular: acompanhar, pausar e ser avisado quando um objetivo termina. O app Android seria editor e também monitor e controle remoto do motor pela rede local. Editar nós em tela pequena é um desafio de UX que combina bem com a disciplina.

5. Casos de uso

5.1 Estudo de caso principal: Hay Day em emulador [Definido]

O primeiro caso real é jogar Hay Day em um emulador no PC. O usuário ensina receitas (por exemplo, pão exige 3 trigos e uma vaga livre na máquina) e o sistema precisa saber localizar as máquinas na fazenda. Depois, ao pedir um sanduíche, ele já sabe chegar até o pão e liga as informações.

O Hay Day foi escolhido porque força quase todos os problemas difíceis de uma vez: estoque, tempo, espaço limitado e posição na tela. Ele acrescentou quatro ideias ao modelo [Proposto]:

- **O estoque vem da tela.** “Tenho 3 trigos?” exige abrir a tela de estoque do jogo e ler os números. Isso também é uma receita e custa tempo, então o motor guarda o último valor lido e só reconfere quando precisa.
- **Item gasta, vaga só ocupa.** O trigo some, mas a vaga da Padaria fica presa até o pão ficar pronto e depois volta. Com poucas vagas, um pedido grande vira várias rodadas.
- **Receita demora.** Fazer pão termina quando o pão fica pronto. O motor aproveita a espera para executar outras receitas e volta depois para recolher.
- **Posição por aparência, não por coordenada.** A câmera se mexe e os prédios podem ser movidos. A receita diz “tocar na Padaria” e o Localizador descobre onde ela está; se algo mudar, a correção é feita em um lugar só. Em testes automatizados, o padrão equivalente é o Page Object.

5.2 Lista de casos de uso

ID	Caso de uso	Resumo	Status
UC01	Cadastrar alvo, cenas e entidades	Capturar a aparência da Padaria, da banca etc. para que as receitas consigam localizá-las	Definido
UC02	Ensinar uma receita	Definir o contrato (consome, ocupa, entrega, onde) e desenhar o fluxo. Ex.: Fazer pão	Definido
UC03	Pedir um objetivo composto	Pedir 1 sanduíche: o planejador inclui “Fazer pão” e “Cultivar trigo” sem religação manual	Definido
UC04	Executar um pedido grande	10 sanduíches com poucas vagas viram várias rodadas; o monitor mostra “esperando o Pão”	Definido
UC05	Manter uma rotina	Colher e vender trigo na banca e coletar as moedas, sem fim	Definido
UC06	Destravar uma pré-condição de cena	“Coletar recompensa diária” exige a cena Menu; o motor vê a cena Batalha e executa “Sair da batalha” antes	Proposto
UC07	Tratar dependência sem produtor	O mapa de receitas marca em vermelho o que nenhuma receita entrega (ex.: Queijo)	Proposto
UC08	Acompanhar e pausar pelo celular	Ver o monitor, pausar rotinas e receber aviso de conclusão	Proposto

ID	Caso de uso	Resumo	Status
UC09	Reutilizar o modelo em outro alvo	O mesmo sistema usado em um programa do Windows ou em um site	Definido

5.3 Fluxo principal: pedir um objetivo composto (UC03 e UC04)

1. O usuário pede “1 Sanduíche” (ou “10 Sanduíches”) como objetivo.
2. O planejador consulta o estado atual, lendo o estoque da tela.
3. Para cada necessidade sem cobertura, busca uma receita que a entregue (Pão → Fazer pão; Trigo → Cultivar trigo).
4. O motor exibe o plano: o que será feito, o que será gasto e o que falta.
5. O usuário confirma e o plano reserva os recursos de que precisa.
6. O executor roda as receitas na ordem, verificando o resultado de cada ação.
7. O monitor mostra o andamento; ao concluir, o usuário é avisado.

Alternativa: se algum item não tem estoque nem receita que o produza, o plano fica bloqueado e mostra exatamente o que falta.

5.4 Fluxo principal: manter uma rotina (UC05)

1. O usuário cria a rotina, definindo gatilho e ação.
2. Um vigia observa o alvo.
3. Quando o gatilho dispara, a rotina pede a vez de agir.
4. O executor faz a ação e confere o resultado.
5. A rotina volta a vigiar; o painel registra a última execução e a ociosidade.

Alternativa: se a rotina quer usar algo reservado por um objetivo, ela só enxerga o que sobrou; se objetivo e rotina querem agir ao mesmo tempo, vale a política de prioridade (seção 8).

6. Catálogo de nós (preliminar)

Rascunho para guiar o editor. A receita completa de “Fazer pão” será usada como molde para detalhar cada nó.

Categoria	Nós previstos	Status
Gatilhos	Quando a cena for X; quando um valor atingir um limite; quando algo ficar pronto; a cada intervalo	Proposto

Categoria	Nós previstos	Status
Observação	Verificar a cena atual; ler uma quantidade (estoque); localizar uma entidade; verificar vaga livre	Definido (verificar o estado)
Ação	Tocar; arrastar; digitar; pressionar tecla; rolar	Proposto
Fluxo	Agir conforme o estado (ramificação); esperar até um estado; esperar um tempo; repetir; tratar falha e tentar de novo	Definido (os dois primeiros)
Orquestração	Executar receita; aguardar receita; supervisionar outras automações até ficarem prontas	Definido
Contrato	Consome; ocupa; entrega; onde	Proposto

Regras de conexão [Proposto, a validar]. No estilo Blueprints, há pinos de execução (a ordem dos passos) e pinos de dados com tipo (quantidade, cena, entidade, verdadeiro ou falso). Só se ligam pinos compatíveis. Todo fluxo de receita começa em um nó de entrada e termina informando sucesso ou falha, para que o planejador saiba o resultado.

7. Requisitos preliminares

7.1 Funcionais

ID	Requisito	Status
RF01	Criar e editar receitas em um editor visual de nós	Definido
RF02	Cadastrar alvos (janela de programa, emulador, aba de navegador)	Proposto
RF03	Cadastrar cenas e entidades, com localizador por aparência	Proposto
RF04	Declarar o contrato da receita (consome, ocupa, entrega, onde)	Proposto
RF05	Verificar o estado do alvo (cena, quantidades, vagas)	Definido
RF06	Agir conforme o estado (ramificação)	Definido
RF07	Esperar até que o alvo atinja um estado	Definido

ID	Requisito	Status
RF08	Supervisionar outras automações até ficarem prontas	Definido
RF09	Resolver dependências automaticamente a partir de um objetivo	Definido
RF10	Exibir o plano (o que fazer, gastar e o que falta) antes de executar	Proposto
RF11	Criar e acompanhar objetivos	Definido
RF12	Pausar e cancelar objetivos	Proposto
RF13	Exibir no monitor o progresso, o estado de cada etapa, a estimativa e a idade da última leitura	Definido
RF14	Criar rotinas (gatilho e ação), ativar e pausar	Definido
RF15	Exibir a ociosidade de cada entidade	Proposto
RF16	Reservar recursos para um plano confirmado	Proposto
RF17	Permitir apenas uma ação de entrada por vez em cada alvo	Proposto
RF18	Acompanhar receitas com duração e aproveitar a espera	Proposto
RF19	Verificar o resultado após cada ação, aplicar timeouts e replanejar	Proposto
RF20	Sinalizar no mapa de receitas o que não tem produtor	Proposto
RF21	Detectar e evitar ciclos entre receitas	Proposto
RF22	Salvar e carregar o projeto em arquivo	Proposto
RF23	Monitorar e controlar o motor remotamente pela rede local	Proposto
RF24	Notificar a conclusão de objetivos e falhas	Proposto

7.2 Não funcionais

ID	Requisito	Status
RNF01	Execução somente em PC	Definido

ID	Requisito	Status
RNF02	Editor disponível em Windows, navegador e Android	Definido
RNF03	Independência do alvo: planejador, monitor e rotinas iguais para qualquer um	Definido
RNF04	Tolerância a estado incerto (atrasos, pop-ups, lag, desconexão)	Proposto
RNF05	Transparência: informar a idade das leituras e explicar por que algo não terminou	Proposto
RNF06	Usabilidade em tela pequena para monitor e rotinas	Proposto
RNF07	Arquivo de projeto portátil e versionado	Proposto
RNF08	Segurança do controle remoto (rede local, pareamento, autenticação)	Proposto
RNF09	Registro de execução (logs) para diagnóstico	Proposto
RNF10	Uso responsável, respeitando termos de uso (seção 9)	Proposto

8. Regras de negócio e políticas do motor

ID	Regra	Status
RN01	**Reserva.** Ao confirmar um plano, ele reserva os recursos que vai usar; as rotinas só enxergam o que sobrou	Proposto
RN02	**Uma mão por alvo.** Só uma ação de entrada por vez em cada alvo; vários vigias podem observar ao mesmo tempo	Proposto
RN03	**Prioridade.** No MVP, o objetivo passa na frente da rotina; prioridade por rotina fica como evolução	Em aberto
RN04	**Estado observado.** O último valor lido é guardado com a sua idade e só reconferido quando necessário	Proposto

ID	Regra	Status
RN05	**Verificação.** Toda ação é seguida de verificação do resultado, com timeout; se falhar, o motor replaneja a partir do que vê	Proposto
RN06	**Consumo e ocupação.** O que é consumido some; o que é ocupado volta ao fim da receita	Proposto
RN07	**Ciclos.** O planejador não pode entrar em loop com receitas inversas	Proposto
RN08	**Escolha de receita.** Quando mais de uma receita entrega o mesmo item, é preciso uma política de escolha (prioridade, custo ou tempo)	Em aberto

9. Riscos, limites e considerações éticas

Risco	Por que importa	Mitigação proposta
Estado observado é incerto	A leitura por imagem, texto ou elementos de interface pode errar, atrasar ou mudar sozinha (pop-up, lag, desconexão)	Verificação após cada ação, timeouts, replanejamento e idade da leitura visível
Só existe uma mão	Mouse e teclado são únicos por alvo; ações concorrentes se atrapalham	Agendador com trava por alvo; instâncias separadas (vários emuladores) como evolução
Ciclos	Receitas inversas formam loop	Detecção no planejador
Conflito entre objetivo e rotina	Uma rotina pode vender o que o objetivo ia usar	Reserva de recursos
Fragilidade visual	Mudanças de aparência ou de posição quebram localizadores	Entidades com localizador editável em um só lugar
Termos de uso	Jogos online costumam proibir automação	Ler os termos do jogo, usar conta de teste e manter o Tomathics genérico
Segurança do controle remoto	Aceitar comandos pela rede pode expor o PC	Rede local, pareamento e autenticação
Escopo grande	Muito projeto para uma disciplina	MVP enxuto (seção 12)

Limites éticos e legais [Proposto]. O Hay Day é um jogo online, e jogos online costumam proibir automação em seus termos de uso. Antes de qualquer teste, é preciso ler os termos do jogo e usar uma conta de teste. O Tomathics é genérico e o jogo é apenas estudo de caso; a ferramenta não se destina a burlar regras ou sistemas anti-trapaça.

10. Referências e similares (rascunho)

Visão inicial: a pesquisa de similares será aprofundada, e as diferenças abaixo são hipóteses a validar.

Referência	Tipo	O que aproveitar	Diferença esperada
Unreal Engine Blueprints	Programação visual por nós	Modelo de interação e pinos	Não automatiza telas de terceiros
Applied Energistics 2	Mod de Minecraft	Resolução automática de dependências e monitor de crafting	No AE2 o estoque é conhecido; aqui ele é observado
STRIPS e GOAP	Planejamento por pré-condições e efeitos	Base teórica do planejador	—
Node-RED	Programação visual por fluxos	Editor de fluxos em nós	Voltado à integração de serviços e dispositivos
UiPath	Automação robótica de processos (RPA)	Automação de interfaces com fluxo visual	Fluxos desenhados à mão; o encadeamento automático por contratos é o diferencial a validar
SikuliX	Automação por reconhecimento de imagem	Localização por aparência	Scripts, sem editor de nós nem planejador
Airtest	Automação de testes de apps e jogos	Automação em emulador e Android com imagem	Foco em teste, não em objetivos e rotinas
Page Object	Padrão de projeto de testes	Isolar o “onde” em um só lugar	Equivale a Entidade + Localizador

11. Arquitetura e tecnologia (adiado)

Por decisão do autor, a tecnologia fica por último. Esta seção registra só o que já está fixado e as perguntas que ficam para depois.

11.1 Restrições já definidas [Definido]

- O motor roda somente em PC.
- O editor existe em Windows, navegador e Android.
- Os alvos são programas do Windows, emulador Android e navegador.

11.2 Módulos lógicos, independentes de tecnologia [Proposto]

Módulo	Responsabilidade
Editor	Criar receitas (nós), cenas e entidades; exibir o mapa de receitas
Biblioteca do projeto	Guardar receitas, cenas, entidades e alvos em um arquivo de projeto versionado (proposta: JSON, compartilhado entre as plataformas)
Percepção	Ler o estado do alvo (cena, quantidades, vagas)
Atuação	Executar ações de entrada no alvo (tocar, digitar etc.)
Planejador	Montar o plano a partir do objetivo e do estado, com reserva e detecção de ciclos
Executor e agendador	Rodar o plano e as rotinas, controlar a “mão” de cada alvo e replanejar
Monitor	Mostrar ao usuário objetivos, rotinas, estado e ociosidade
Comunicação remota	Permitir que o editor e o monitor (web e Android) falem com o motor no PC

11.3 Decisões tecnológicas pendentes [Em aberto]

- Como ler a tela de cada tipo de alvo (imagem, reconhecimento de texto, árvore de elementos de interface).
- Como controlar cada alvo (entrada simulada, ponte com o emulador, automação de navegador).

- Tecnologia do editor multiplataforma e do app Android, conforme as exigências da disciplina.
- Protocolo de comunicação entre o app e o motor.
- Formato final do arquivo de projeto.

12. Escopo do MVP e roadmap [Proposto]

O MVP abaixo é uma proposta e depende das exigências da disciplina (prazo, entregáveis e tecnologia obrigatória).

MVP candidato: um único alvo (Hay Day no emulador, como estudo de caso), com receitas com contrato, planejador simples (sanduíche → pão → trigo), executor com verificação, monitor básico e rotinas simples, tudo com o objetivo passando na frente da rotina.

Fase	Entrega	MVP
0	Concepção e documentação (em andamento)	Sim
1	Modelo e editor: formato de receita e contrato, editor de nós básico, mapa de receitas	Sim
2	Percepção e atuação em um alvo: ler cena e quantidades, tocar em entidades, executar uma receita de ponta a ponta	Sim
3	Planejador e monitor: objetivos com dependências, pré-visualização do plano, monitor de objetivos	Sim
4	Rotinas e agendador: gatilhos, reserva de recursos, objetivo antes de rotina	Sim
5	App Android: editor, monitor e controle remoto pela rede local	Provável, por ser o núcleo mobile da disciplina
6	Outros alvos (programa do Windows, navegador), prioridade por rotina e vários alvos ao mesmo tempo	Não

13. Pontos para alinhar

#	Ponto	Opções	Recomendação inicial
1	Prioridade entre objetivo e rotina quando disputam a mesma “mão”	(a) o objetivo sempre passa na frente; (b) prioridade por rotina (ex.: “coletar moedas” nunca espera)	(a) no MVP e (b) como evolução
2	Reserva de recursos	Rotinas só enxergam o que sobrou do plano, ou rotinas livres	Reserva, para não vender o trigo do objetivo
3	Vocabulário	Confirmar ou trocar: Receita, Objetivo, Rotina, Alvo, Entidade, Localizador, Plano, Executor	Manter por ora e revisar pelo glossário
4	Exigências da disciplina	Prazo, entregáveis, tecnologia obrigatória, artefatos (UML, requisitos)	Levantar antes de fechar o MVP
5	Papel do app Android	Só editor, ou editor e também monitor e controle remoto	Editor e monitor, por ser o núcleo mobile
6	Sistema operacional do motor	Somente Windows ou também outros	Windows primeiro
7	Vários alvos ao mesmo tempo	Um alvo por vez ou vários (ex.: vários emuladores)	Um por vez no MVP
8	Receitas que entregam o mesmo item	Prioridade manual, menor custo ou menor tempo	Prioridade manual no MVP
9	Granularidade das receitas	Uma receita por resultado (Fazer pão) ou por ação	Por resultado
10	Posicionamento ético	Jogo só como estudo de caso, conta de teste, sem burlar anti-trapaça	Adotar e manter registrado

#	Ponto	Opções	Recomendação inicial
11	Leitura do estoque	Reler a cada necessidade ou guardar e reconferir	Guardar com a idade e reconferir quando necessário

14. Próximos passos

1. Alinhar os pontos da seção 13.
2. Detalhar a receita “Fazer pão” por inteiro (fluxo, verificações e falhas) como molde do catálogo de nós.
3. Produzir diagramas: casos de uso (UML), arquitetura lógica e fluxo do planejador.
4. Aprofundar a pesquisa de similares.
5. Fechar o MVP conforme as exigências da disciplina.
6. Só então definir a tecnologia.

Histórico de versões

Versão	Data	Mudança
0.1	30/10/2026	Consolidação inicial da ideação: visão, modelo conceitual, telas, casos de uso, requisitos preliminares e pontos em aberto